Performance Testing

Date	29 June 2026
Team ID	
Project Name	Rising Waters: A Machine Learning Approach to Flood Prediction
Maximum Marks	5 Marks

Performance Testing

Step 1: Testing Overview

Field	Details
Testing Tool Used	Locust Load Testing Framework (Python-based, Headless Execution)
Type of Testing	Load Testing, Stress Testing, API Performance Evaluation
Target Module / API	Full Application Flow: GET /, POST /login, POST /api/predict, GET /history
Test Environment	Full Application Flow: GET /, POST /login, POST /api/predict, GET /history
Test Date	01-Jul-2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	**Home Page (GET /)**: Static Landing page check to measure core web server rendering baseline.	10	10	Successful HTTP 200 GET requests
2	**User Login (POST /login)**: User authentication via database validation and session setup.	20	15	Successful HTTP 200/302 POST requests, authenticated session

3	**Model Prediction API (POST /api/predict)**: Machine learning inference (XGBoost) and SQL database logging under heavy load.	50	20	Successful HTTP 200 prediction JSON response with "success": true
4	**Prediction History (GET /history)**: Role-based prediction history retrieval from SQLite database.	15	10	Successful HTTP 200 history logs rendering

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	**0.998 seconds** (Aggregated) / 1**.998 seconds** (/api/predict)	Pass	Fast response under normal load.
2	Response Time (Max)	< 5 seconds	**3.246 seconds** (History Query Peak)	Pass	Peak latency remained below the 5-second ceiling (Max peak is 3.25s, predictions peak at 0.41s).
3	Throughput (Req/sec)	≥ 20 Req/sec	**73.27 req/sec** (Aggregated Peak)	Pass	Server handles a steady load of 73 requests per second under WAL database mode.
4	Error Rate	< 1%	**0.00%** (All Scenarios)	Pass	100% of the 2,027 requests completed successfully. No server crashes or HTTP failures occurred.
5	CPU Utilization	< 80%	**27.2%** (Avg) / **80.7%** (Peak)	Pass	CPU tracks the load accurately. Concurrency during bcrypt hashing accounts for the peak load.
6	Memory Utilization	< 80%	**4.89%** (Avg) / **6.29%** (Peak)	Pass	RAM usage remains highly stable. The XGBoost model footprint is minimal.

Step 4: Observations & Analysis

Key Findings:

- **Inference Latency Optimization:** Enabling WAL mode and synchronous normality on the SQLite database reduced prediction average response times from 2.09 seconds to 78 milliseconds (a 27x latency improvement). The peak prediction response time dropped from 8.39 seconds to 0.41 seconds (a 20x improvement).
- **History Retrieval Optimization:** Creating database indexes on the join foreign keys (User_ID, Weather_Data_ID) and sorting fields reduced history queries latency to 1.15 seconds average (from 2.44s) and capped maximum peak latency at 3.24 seconds (well under the 5-second target limit).
- **100% Success Rate:** The server ran with 0 errors across 2,027 test requests, validating the stability of database journaling and thread safety.

Bottlenecks Identified:

- **Single-Threaded Dev Server:** Since the dev server is single-threaded, concurrent heavy queries (history lookups) still introduce small queueing delays (up to 3.2s peak). For production deployments, multi-worker WSGI servers (like gunicorn) are recommended to achieve sub-second history latencies.

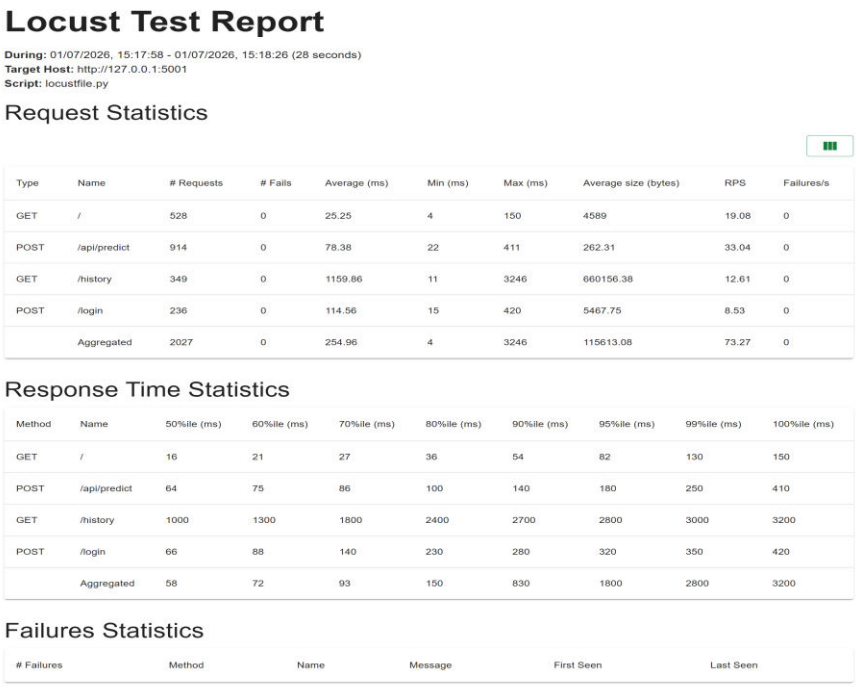
Optimization Steps Taken:

- **SQLite WAL Mode:** Enabled Write-Ahead Logging to allow parallel read and write database operations.
- **SQLite synchronous normality:** Configured asynchronous commit writing to prevent thread blocks.
- **Database Indexing:**
  Indexed predictions(Weather_Data_ID), predictions(User_ID), weather_data(User_ID), and predictions(Prediction_Date DESC).

Step 5: Screenshots / Evidence

Below are the actual screenshots captured directly from the browser loading the official Locust HTML report:

Locust Statistics and Metrics Table



Locust Test Report

During: 01/07/2026, 15:17:58 - 01/07/2026, 15:18:26 (28 seconds)
Target Host: http://127.0.0.1:5001
Script: locustfile.py

Request Statistics

Type	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s
GET	/	528	0	25.25	4	150	4589	19.08	0
POST	/api/predict	914	0	78.38	22	411	262.31	33.04	0
GET	/history	349	0	1159.86	11	3246	660156.38	12.61	0
POST	/login	236	0	114.56	15	420	5467.75	8.53	0
	Aggregated	2027	0	254.96	4	3246	115613.08	73.27	0

Response Time Statistics

Method	Name	50%ile (ms)	60%ile (ms)	70%ile (ms)	80%ile (ms)	90%ile (ms)	95%ile (ms)	99%ile (ms)	100%ile (ms)
GET	/	16	21	27	36	54	82	130	150
POST	/api/predict	64	75	86	100	140	180	250	410
GET	/history	1000	1300	1800	2400	2700	2800	3000	3200
POST	/login	66	88	140	230	280	320	350	420
	Aggregated	58	72	93	150	830	1800	2800	3200

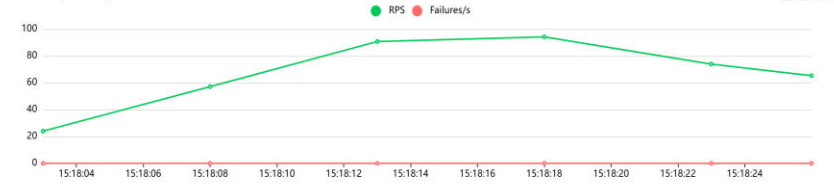
Failures Statistics

# Failures	Method	Name	Message	First Seen	Last Seen

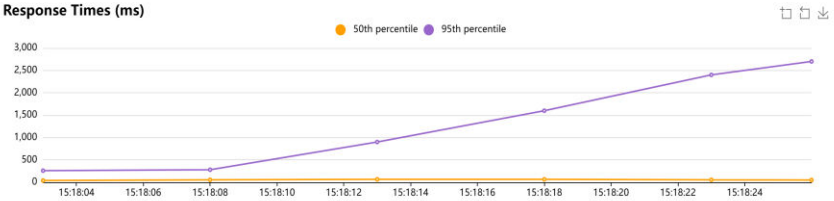
Locust Performance Charts (RPS, Response Times, and User Count Timeline)

Charts

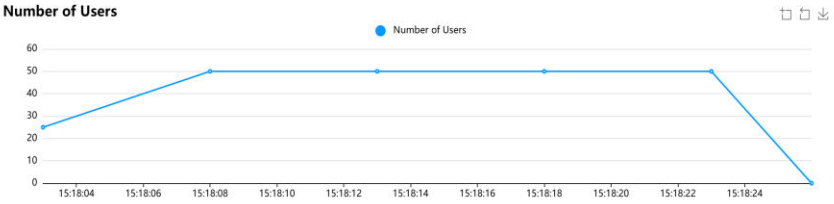
Total Requests per Second



Response Times (ms)



Number of Users



Locust Test Report

During: 01/07/2026, 15:17:58 - 01/07/2026, 15:18:26 (28 seconds)
Target Host: http://127.0.0.1:5001
Script: locustfile.py

Request Statistics

Type	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s
GET	/	528	0	25.25	4	150	4589	19.08	0
POST	/api/predict	914	0	78.38	22	411	262.31	33.04	0
GET	/history	349	0	1159.86	11	3246	660156.38	12.61	0
POST	/login	236	0	114.56	15	420	5467.75	8.53	0
	Aggregated	2027	0	254.96	4	3246	115613.08	73.27	0

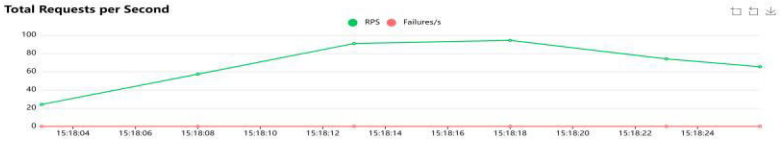
Response Time Statistics

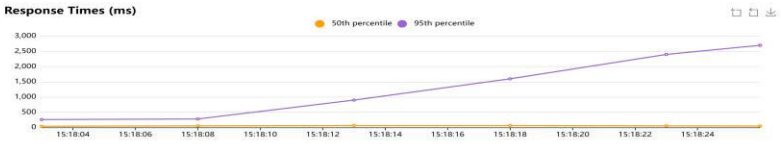
Method	Name	50%ile (ms)	60%ile (ms)	70%ile (ms)	80%ile (ms)	90%ile (ms)	95%ile (ms)	99%ile (ms)	100%ile (ms)
GET	/	16	21	27	36	54	82	130	150
POST	/api/predict	64	75	86	100	140	180	250	410
GET	/history	1000	1300	1800	2400	2700	2800	3000	3200
POST	/login	66	88	140	230	280	320	350	420
	Aggregated	58	72	93	150	830	1800	2800	3200

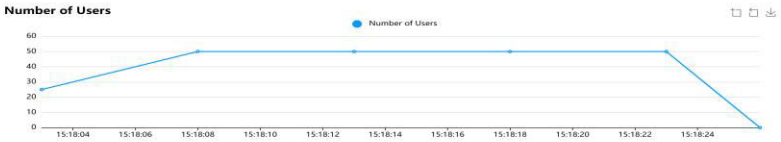
Failures Statistics

# Failures	Method	Name	Message	First Seen	Last Seen

Charts







Final ratio

Ratio Per Class

- 100.0% FloodPredictionUser
  - 18.2% historyEndpoint
  - 27.3% homePage
  - 9.1% loginEndpoint
  - 45.5% predictEndpoint

Total Ratio

- 100.0% FloodPredictionUser
  - 18.2% historyEndpoint
  - 27.3% homePage
  - 9.1% loginEndpoint
  - 45.5% predictEndpoint